PITFALL: Catching Point-in-Time Violations in Multi-Table Feature Pipelines by Differential Execution

Stanislav Chumakov, Danil Ezhov, Alexander Boukhanovsky
AI Institute, ITMO University, Saint Petersburg, Russian Federation
{svchumakov, doezhov}@itmo.ru

***Abstract*—In multi-table prediction tasks, features are typically aggregates over neighbouring tables computed as of a per-row seed time: a seller's average review so far, orders in the last month. Such an aggregate must read only what existed at that seed time, along every join path and for every column, including columns written to a row after the row appears. An aggregate that reads later data looks fine offline, where past seed times have a future in the database, and differs from what exists when the prediction is made. We find violations of this requirement in five sources we did not construct: a feature-engineering library called as in its own introductory example, our own reference code, code generated by two language models, published expert SQL from a benchmark's user study (8 of the 14 files that execute), and SQL written by an LLM agent. Inflation reaches 16 AUC points in the sources we found and 27 under a controlled cutoff shift, and depends on the task more than on the defect: our largest violation, hundreds of thousands of diverging cells, costs nothing measurable. We present PITFALL, which runs a feature program twice, on the full database and on one truncated at the seed time, and reports any divergence. A divergence is a witness of a violation under the declared availability map; the check does not parse the program, names the offending columns in seconds, and localises the leak by truncating one availability channel at a time. The univariate probe that commercial AutoML platforms deploy against leakage fails in both directions: silent on every violation we report at DataRobot's default threshold, firing on correct pipelines at H2O's. Code and data: https://github.com/stas1f1/pitfall; interactive demo: https://stas1f1.github.io/pitfall/.**

***Index Terms*—data leakage, point-in-time correctness, relational machine learning, AutoML, feature engineering, LLM-generated code**

I. Introduction

Consider one seller on a marketplace (Fig. 1). An order is placed on 5 March and reviewed on 5 April. Predicting something about the seller *as of* 1 April, a feature may use the order but not the review, although the review is attached to a row that already existed. A filter on the order date admits it. We catch this by running the same feature program twice, once on the full database and once on a copy from which everything dated after 1 April has been removed, and comparing the outputs. If they differ, the program used information that did not exist at prediction time; it peeked. The feature here is an aggregate, the seller's average review as of a date. In training, the database already holds what came after that date; when the prediction is actually made, it does not, so a peeking aggregate looks good offline and differs from what the model will see in use. We call the property it violated *point-in-time correctness*: every feature computed as of a seed time t may only read facts that existed at t.

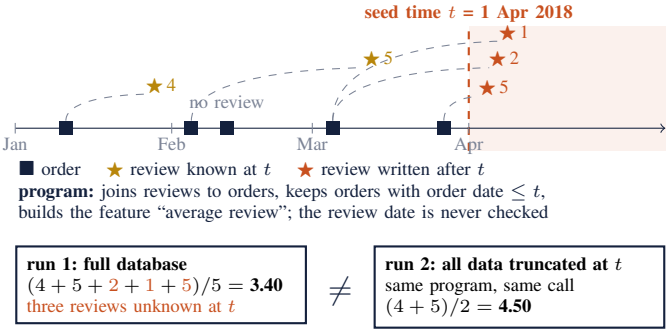

Fig. 1. One seller from the Olist database; three of the five reviews were written after the seed time. PITFALL does not infer leakage from code or from feature statistics: it tests whether the executed program depends on information that was unavailable at prediction time, by running it twice and comparing.

In multi-table (relational) machine learning a task is a table of *(entity, seed time, label)* triples over a database with foreign keys and timestamps [8], and features are aggregates over join paths out of the entity. It is easy to violate: truncation is per row, so one cut at the start of the test period says nothing about individual seed times; it must hold along every join path, where the governing timestamp often belongs to another table; and a row can carry columns written later than the row itself, like the review above, which a filter on the row's own timestamp lets through. The third case is the one most often missed.

Leakage is a documented driver of irreproducibility in applied ML [1], [2], and existing safeguards do not cover this case. Commercial AutoML platforms (DataRobot, H2O Driverless AI, SageMaker) all ship one detector, which we call the *univariate probe*: the highest AUC any single feature reaches on its own, flagged above a threshold, Gini Norm 0.85 (AUC 0.925) for DataRobot and AUC 0.80 for H2O [6], [7]. Notebook analyzers [3] and pipeline instrumentation [4] target preprocessing order and train/test contamination and do not cover temporal leakage; asking a language model directly reaches 0.19 precision at 0.52 recall [5].

Contributions. (i) An execution-based check for feature programs in the spirit of metamorphic testing: one-sided, language-agnostic, with per-column availability and leak localisation by per-channel truncation. (ii) Evidence from five

sources on six public databases (Table I), including an execution-verified rate at which two language models write temporally incorrect feature code and a demonstration that timestamp granularity decides whether a boundary error costs anything. (iii) An interactive demonstration in which a visitor catches the bug, sees why the industrial probe misses it, and localises it.

II. Point-in-Time Correctness

A. Definition

Let D be a database, t a seed time and φ a feature program. Each row r has a row timestamp $\mathrm{time}(r)$; a column c of r may in addition have its own availability timestamp $\mathrm{avail}_c(r) \geq \mathrm{time}(r)$. Write $D|_t$ for the database in which rows with $\mathrm{time}(r) > t$ are removed and values with $\mathrm{avail}_c(r) > t$ are replaced by NULL. Then φ is *point-in-time correct at* t if and only if

$$\varphi(D, t) = \varphi(D|_t, t). \tag{1}$$

In the code the boundary is a *cutoff*; a shift $\delta \geq 0$ means the program reads up to $t + \delta$. $I(\delta)$ is the AUC of a *fixed* learner on features computed with cutoff $t+\delta$ minus its AUC at $\delta = 0$.

B. Availability relation and channels

Equation (1) generalises by replacing the time predicate with an *availability relation* $R(r, e, t)$, true when row r may be read when predicting for entity e at time t. Availability is per column: in our data `review_score` becomes available at `review_creation_date`, a median of 10 days after the order row (maximum 147). The database therefore has a finite set of *availability channels*: rows of table T appearing after t, and values of column c becoming known after t. Some columns have no availability timestamp at all, e.g. a mutable status field kept without history; the truncated database is then identical to the full one and no execution-based check can decide them (we exclude `order_status` throughout).

C. Differential execution, guarantee, localisation

The check follows from (1): run φ twice and compare:

```
program(db,t,ents) !=
program(db.truncate(t),t,ents)
```

A detected divergence is a witness of a point-in-time violation under the declared availability map: the same program, given strictly less information, produced a different answer. The test is one-sided: no divergence at the tested seed times does not prove correctness, and the program must be deterministic. With floating-point aggregates a tolerance is required, because truncation changes the query plan and summation is not associative; we set it from the noise floor of a negative control (1.7×10^{-11} relative); without it four verdicts on rel-amazon would have been false.

The same primitive localises the leak. Truncating one channel at a time identifies the channels on which the output changes and the output columns each affects; masking exactly those channels and re-checking confirms that no other channel contributes. Masking diagnoses without repairing: any program passes on a pre-truncated input, while in training tables truncation must happen per row inside the program, which is what the check verifies.

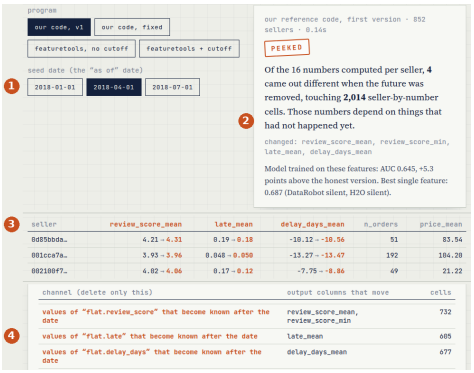

Fig. 2. The demonstration page. **1** choose the seed date; **2** choose a program and run it twice: verdict, diverging columns, inflation of the fixed model, and what the univariate probe would say; **3** outputs side by side, full database → past only; **4** localisation: each channel removed alone and the output columns that move (silent channels omitted).

III. System

PITFALL sits between any feature builder and any tabular AutoML. Scout holds the availability map (which column governs the availability of which field, and which fields have none); today it is a small hand-written declaration. Builder is any producer of a feature program: library, agent or person. Guard runs the differential check and returns the verdict with the diverging columns and cells; on a violation Locator runs the per-channel truncation. Clean programs go to an unmodified tabular AutoML. The core is ∼170 lines of Python over pandas. **Validation.** On six reference programs with known verdicts at three seed times the checker was right on 18 of 18 combinations. A negative control (seed time past every timestamp) caught three defects of our own before the measurements that depended on them.

IV. Demonstration

The demonstration is a self-contained interactive page built from live runs (Fig. 2), plus a console runner, `python3 demo.py`. At the stand it runs in three acts of about twenty seconds each.

Act 1, catch the bug. One real seller and a draggable date (the data of Fig. 1). As the date moves, the naïve "average review" and the average of the reviews actually written by that date diverge, and the stars written after the line turn red: the order existed, its review did not.

Act 2, why not just look at the feature. "Guess the peek": eight short feature functions over the same table; the visitor says which ones peek, then the check answers. Average review looks safe and is not; the z-score of price peeks with no join at all, through a statistic fitted on the whole table. The verdict card then shows what the univariate probe says about the leaking program: 0.69 on task B, silent at both deployed thresholds, while the check says violation and the fixed model gains 5.3 points.

TABLE I
FIVE SOURCES OF POINT-IN-TIME VIOLATIONS, NONE CONSTRUCTED BY US. VERDICTS TAKE SECONDS PER PROGRAM ON OLIST AND REL-F1; THE CORPORA ARE NOT MUTUALLY COMPARABLE; RANGES SPAN THE THREE SEED TIMES.

source	run / diverge	ΔAUC, pp
`featuretools` 1.31 default, Olist C	1 / 1, 3 seeds	$[+14.8; +16.3]$
our reference code, Olist A and B	1 / 1, 3 seeds	A $+0.2$; B $[+3.1; +5.3]$
LLM Python, 2 models, Olist C	132 / 61 of 233	median $+0.11$
expert SQL (RelBench study), 5 DBs	14 / 8 of 15 files	-0.33, $+0.95$, -0.02
LLM-agent SQL, rel-stack	34 / 2 of 37	not measured

Act 3, where did it leak. Channels are removed one at a time. Two review aggregates move only when review scores written after the date are hidden, two delivery aggregates only when the delivery date is hidden; nothing else in the database matters to this program. The tool locates; the repair is two lines, shown as "our code, fixed", which is clean.

V. DATA AND SETUP

Olist [11] is a public Brazilian marketplace database, 7 tables and 112,650 order line items, on which we define three tasks in RelBench [8] form: seller activity (A), seller review quality (B) and product demand (C), at three seed times in 2018 with a 90-day horizon. **rel-f1** [8] is the Ergast Formula 1 database, 9 tables and 26,080 driver-race results over 1950–2023; our task is *will this driver retire from the next race*, seed time at midnight of race day. **Corpus databases** (Section VI-C): rel-f1, rel-stack, rel-event, rel-hm and rel-amazon, 15 tasks, tables of up to 10^8 rows, availability from RelBench's `time_col` declarations. **Baseline, model and intervals.** The baseline for every cost figure is the same program run under the correct cutoff, with one LightGBM configuration and a fixed seed everywhere; inflation is the paired AUC difference against it, with percentile bootstrap intervals on the same test rows (2,000 resamples). The baseline detectors are the univariate probe at both deployed thresholds, the manual audit and our earlier static scan (Table II). **Runs.** Olist: 3 tasks $\times$ 3 seed times, plus 54 cutoff-shift runs (9 values of δ, tasks A and B); rel-f1: one task, 6 test cells of 1,042–1,203 rows, three per timestamp era; expert SQL: 15 files $\times$ 3 seed times (5 on rel-event); agent corpus: 37 queries $\times$ 2 seed times, each with a determinism rerun and a negative control; 233 LLM generations.

VI. EVIDENCE

A. The cost of a shifted cutoff

$I(\delta)$ increases monotonically over $\delta \in [0, 90]$ days on Olist tasks A and B at all three seed times: a one-month error costs $[5.8; 11.5]$ points, no cutoff at all up to $+27.0$ (A) and $+18.0$ (B). On task C, controlling the cutoff separately for the entity's own history and for the join to the seller, leaking only through the join costs $[+3.0; +5.1]$ points and does not move the probe at all (Table II).

B. What $\leq t$ instead of $< t$ costs depends on the timestamps

Whether the cutoff comparison matters is decided by the timestamps, and rel-f1 settles this inside one database: race stamps carry no time of day before 2005 and carry the start time from 2005 on. An outcome is available strictly after the race starts, so the inclusive cutoff $\leq t$ admits the race being predicted in the first era and nothing in the second. In 0.02 s per seed time, before any model is fitted, the checker diverges on every day-stamped seed time and is clean on every second-stamped one. The same inclusive cutoff then costs -0.45, $+2.45$ and $+3.27$ points on the day-stamped cells and exactly 0.00 on all three second-stamped cells. On Olist, where events carry seconds, it also cost 0.00; without rel-f1 we could not have told whether that generalises.

C. Sources of violation

Table I lists the five sources. `featuretools` [9] with `ft.dfs` and no cutoff-time table, the form of its introductory example [10], supports cutoff times but does not say they are needed: 11 columns diverge, fed by `items` rows after t. Our own reference code filtered history by order time and took all columns of the admitted rows, including the two with their own later timestamps; the checker flagged exactly the four review and delivery aggregates, 2,014 cells.

Published expert SQL, executed. The hand-written feature SQL of the RelBench user study [8] is 15 files over 172 joins, run unchanged. Fourteen files execute; **8 diverge** and 6 are clean at every seed time. The three rel-f1 files read `races` rows dated after the seed time; `stack/user-badge` takes a global frequency over `badges` with no cutoff, the violation the audit did find; on `stack/post-votes`, which the audit cleared, the output depends on 2,730 `users` rows dated after the posts they own. The file that does not execute reads `posts.AcceptedAnswerId`, a mutable field with no history that RelBench itself deletes as a "time leakage column", our Section II-B verdict.

How much a clean verdict is worth. The three rel-event files join `users` without a cutoff, so six demographic columns read rows of users who registered after the seed time. The violation reaches 87.5% of label rows and is present at 23 of the task's 24 seed times and absent at exactly one, RelBench's own test seed time. That is no coincidence: data beyond a seed time only shrinks as the seed time grows. Seed times must come from the middle of the training split.

An agent-written corpus. The 37 distinct queries an LLM agent wrote across 32 trials on `stack/user-engagement` had only a static scan with 5 candidates. Running them takes 19 minutes: 34 execute, **2 diverge**, both the same missing cutoff on the join to `users`, giving account ages down to $-2{,}021$ days on 57 of 25,233 label rows. Against execution the scan errs both ways: 3 of its 5 candidates are false and 2 real.

Generated feature code. We used `deepseek-v4-flash` and `bytedance-seed/seed-2.0-code` to generate a Python function computing features for task C, giving the schema and the seed-time column, not mentioning "leakage" or "point-in-time"; verdicts come from the checker, with five flagged programs manually confirmed. Of 233 generations

TABLE II

BASELINE DETECTORS COMPARED. (A) THE 14 EXECUTED EXPERT-SQL FILES, 8 OF WHICH DIVERGE; GROUND TRUTH IS DIVERGENCE UNDER EXECUTION, SO PITFALL IS CORRECT BY CONSTRUCTION THERE. (B) CONTROLLED RUNS ON OLIST (TASKS A–C) AND REL-F1; "BY ERA": VIOLATION ON DAY-STAMPED SEED TIMES, CLEAN ON SECOND-STAMPED ONES.

(a) expert SQL, 14 files, 8 violations

detector	result
manual audit [8]	2 found, 1 of the 8; cleared `post-votes` (diverges)
static scan (ours, earlier)	8 candidate joins, 2 real, 6 false
univariate probe	identical to 6 s.f. with and without the violation
PITFALL	**8 of 8**, each traced to the table and column

(b) the probe fails in both directions (**bold** = wrong call; ranges: three seed times)

run	ΔAUC, pp	probe	DataRobot / H2O	PITFALL
correct code, A	0	[0.83; 0.86]	silent / **false alarm**	clean
join leak only, C	[+3.0; +5.1]	0.71 both	**miss** / **miss**	violation
featuretools, C	[+14.8; +16.3]	[0.76; 0.83]	**miss** / fires (2/3)	violation
reference code, B	[+3.1; +5.3]	[0.62; 0.69]	**miss** / **miss**	violation
no cutoff, B	+18.0	[0.80; 0.83]	**miss** / fires (2/3)	violation
$\leq t$ vs $< t$, rel-f1	[−0.45; +3.27]	moves ≤ 0.015	silent / same	by era

TABLE III

THE SIZE OF A VIOLATION DOES NOT PREDICT ITS COST; THE TASK AND THE TIMESTAMPS DO.

case	diverging cells	inflation, pp
our reference code, Olist task A	4 columns, 2,014 cells	+0.2
our reference code, Olist task B	the same cells	[+3.1; +5.3]
expert SQL, `f1/driver-top3`	`races` rows after t	+0.95
expert SQL, `f1/driver-dnf`	`races` rows after t	−0.33
expert SQL, `stack/user-badge`	9 columns, 345,938 cells	−0.02

132 ran and 61 were incorrect, 62% for the first model and 22% for the second; since roughly half the generations crash before producing a feature table, the rate over all 233 lies in [26%; 70%]. Both models fail the same way, filtering the entity's own history by the primary event timestamp and ignoring the later timestamp of the joined one. We make no claim about models in general. Task construction is decisive: on a second task with one seed time per entity and no secondary time axis, both models produce 0 violations. Prompt guards help partially but not to zero: a reminder and then a per-aggregate requirement cut the pooled rate from 46.2% to 41.2% to 14.6%.

D. What the univariate probe reports

Table II compares the baseline detectors. No value of the probe separates leaking runs from clean ones: leakage through a join does not move it at all, on task B nothing reaches DataRobot's threshold, not even the complete absence of a cutoff, and H2O's lower threshold fires on the correct task-A pipeline at every seed time, because recency is a legitimate predictor of churn.

E. Cost of a violation is task-dependent

The same defect in the same code costs [3.1; 5.3] points on task B, while on task A every interval contains zero (Table III); among model-generated programs the median inflation is 0.11 points. The expert SQL makes the same point on code we did not write: rebuilt on a per-row truncated database and refitted with the same model, its violations cost −0.33, +0.95 and −0.02 points, the last over 3×10^5 diverging cells. Neither the fact of a violation nor its size predicts its cost, so correctness has to be tested separately from the score.

VII. LIMITATIONS

(i) One-sided. The check proves violation and cannot prove correctness, and only with respect to the observed database, the chosen mask and the seed times tested. *(ii) Undecidable columns.* Mutable fields kept without history cannot be checked by execution. *(iii) The availability map.* A wrong timestamp means the wrong thing is checked, and the map is a modelling decision; the check decides code against the declared relation only. *(iv) Scope.* Cost is measured on two databases and four tasks; the corpus verdicts span five more databases but carry no cost. We measure how much a violation inflates the offline estimate (train and test both leaking against both correct); the further loss from training on leaking features and serving on correct ones, where the future does not yet exist, is not measured here. *(v) Determinism.* Columns that differ between identical reruns carry no verdict. *(vi)* Validation-driven trial selection can discard a leaking intermediate trial, so final-program counts undercount (observed once).

VIII. CONCLUSION

Point-in-time violations occur in library defaults, published expert code and generated code; the univariate probe misses the regimes reported here. Their cost is not predictable from their existence or size, so correctness has to be tested directly, and the score does not show it. A clean verdict covers only the seed times that were tested; at other seed times the same program can still peek, as the rel-event files show.

DATA, ETHICS AND AVAILABILITY

Experiments use public datasets: Olist (Kaggle, CC BY-NC-SA 4.0) and the rel-f1, rel-stack, rel-event, rel-hm and rel-amazon databases with the expert SQL released alongside RelBench [8]; no human subjects. Code, data, unedited model outputs and all reported numbers: https://github.com/stas1f1/pitfall.

REFERENCES

[1] S. Kaufman, S. Rosset, and C. Perlich, "Leakage in data mining: formulation, detection, and avoidance," in *Proc. KDD*, 2011.

[2] S. Kapoor and A. Narayanan, "Leakage and the reproducibility crisis in machine-learning-based science," *Patterns*, vol. 4, no. 9, 2023.

[3] C. Yang *et al.*, "Data leakage in notebooks," in *Proc. ASE*, 2022.

[4] S. Grafberger *et al.*, "mlinspect: a data distribution debugger for ML pipelines," in *Proc. SIGMOD*, 2021.

[5] N. Alturayeif and J. Hassine, "Data leakage detection in ML code," *PeerJ Computer Science*, 11:e2730, 2025.

[6] DataRobot documentation, "Data quality assessment: target leakage," docs.datarobot.com, 2026.

[7] H2O.ai, Driverless AI 1.11 User Guide, "Data leakage and shift detection," docs.h2o.ai, 2026.

[8] J. Robinson *et al.*, "RelBench: a benchmark for deep learning on relational databases," *NeurIPS D&B*, 2024.

[9] J. M. Kanter and K. Veeramachaneni, "Deep feature synthesis," in *Proc. DSAA*, 2015.

[10] Featuretools 1.31 documentation, "Handling time," featuretools.alteryx.com, 2026.

[11] Olist, "Brazilian e-commerce public dataset," Kaggle, 2018.